```
window.liveTimeEnabled = false; const canvas = document.getElementById("canvas"), ctx = canvas.getContext("2d"); ctx.imageSmoothingEnabled = false; const offscreenCanvas = document.createElement('canvas'), offscreenCtx = offscreenCanvas.getContext('2d', { alpha: false }); offscreenCtx.imageSmoothingEnabled = false; let worldDirty = true, lastCameraX = null, lastCameraY = null, lastGridWidth = null; const uiElements = { topBar: document.getElementById('top-bar'), ribbon: document.getElementById('ribbon') }; let currentZoom = 100, showGrid = false, tileSize = 16; const BASE_TILE_SIZE = 16, grid = { width: 60, height: 30 }, camera = { x: 0, y: 148, speed: 1 }; window.images = { names: ["blocks", "hotbar", "slot", "nether-bg", "end-bg", "zombie", "skeleton", "creeper", "enderman", "nethereye", "enderdragon", "pig", "cow", "chicken", "Player", "slime", "magmacube", "chicken", "blaze", "squid", "ghast", "rabbit", "cowctus cow", "mushroom cow", "dog", "sheep", "snowgolem", "wolf", "spider", "snowgolem", "zombiepigman", "bat", "zombie_supremo"] }; window.images.names.forEach(name => { window.images[name] = new Image(); window.images[name].src = `assets/${name}.png`; window.images[name].onload = () => worldDirty = true; }); function resizeCanvas() { const ws = document.getElementById('workspace'); if (ws) { canvas.width = ws.clientWidth; canvas.height = ws.clientHeight; } else { canvas.width = window.innerWidth - 60; canvas.height = window.innerHeight - 112; } offscreenCanvas.width = canvas.width; offscreenCanvas.height = canvas.height; offscreenCtx.imageSmoothingEnabled = false; ctx.imageSmoothingEnabled = false; updateGridDimensions(); worldDirty = true; } function updateGridDimensions() { tileSize = BASE_TILE_SIZE * (currentZoom / 100); grid.width = Math.ceil(canvas.width / tileSize); grid.height = Math.ceil(canvas.height / tileSize); camera.speed = 100 / currentZoom; camera.x = Math.round(camera.x); camera.y = Math.round(camera.y); worldDirty = true; } window.addEventListener('resize', resizeCanvas); window.onload = resizeCanvas; window.updateZoomPreview = val => { const t = document.getElementById('zoom-value'); if (t) t.innerText = val + "%"; }; window.applyZoomExact = val => { currentZoom = Math.max(2, Math.min(600, parseInt(val))); updateZoomPreview(currentZoom); const s = document.getElementById('zoom-slider'); if (s) s.value = currentZoom; updateGridDimensions(); }; const statusRightZone = document.querySelector('.status-right'); if (statusRightZone) { statusRightZone.addEventListener('wheel', e => { e.preventDefault(); let step = currentZoom < 50 ? 5 : (currentZoom < 200 ? 10 : 25); applyZoomExact(currentZoom + (e.deltaY < 0 ? step : -step)); }); } function toggleGrid(enabled) { showGrid = enabled; worldDirty = true; } window.showDimBackgrounds = true; window.toggleDimBackgrounds = enabled => { window.showDimBackgrounds = enabled; worldDirty = true; }; function initializeWorldCache() { window.worldCache = []; if (typeof mbwom !== 'undefined' && mbwom.scene && Array.isArray(mbwom.scene)) { for (let x = 0; x < mbwom.scene.length; x++) if (mbwom.scene[x]) for (let y = 0; y < mbwom.scene[x].length; y++) renderBlock(x, y); } worldDirty = true; } function renderBlock(x, y) { if (!worldCache[x]) worldCache[x] = []; const states = mbwom.getBlockState(x, y); if (states.type != null) worldCache[x][y] = getBlockObject(states); else delete worldCache[x][y]; worldDirty = true; } function drawBlock(texture, values, targetCtx = ctx) { if (window.images.blocks?.complete && window.images.blocks.naturalWidth !== 0) targetCtx.drawImage(window.images.blocks, texture.x, texture.y, 16, 16, values.x, values.y, values.width, values.height); } function renderWorldToBuffer() { offscreenCtx.fillStyle = "#778fa5"; offscreenCtx.fillRect(0, 0, offscreenCanvas.width, offscreenCanvas.height); let cS = typeof mbwom !== 'undefined' ? mbwom.currentScene || 1 : 1; if (window.showDimBackgrounds) { if (cS === 2 && window.images['nether-bg']?.complete) offscreenCtx.drawImage(window.images['nether-bg'], 0, 0, offscreenCanvas.width, offscreenCanvas.height); else if (cS === 3 && window.images['end-bg']?.complete) offscreenCtx.drawImage(window.images['end-bg'], 0, 0, offscreenCanvas.width, offscreenCanvas.height); } for (let x = 0; x < grid.width; x++) { for (let y = 0; y < grid.height; y++) { const bO = getBlockCache(Math.floor(x + camera.x), Math.floor(y + camera.y)); if (bO != null) drawBlock(bO, { x: Math.floor(x * tileSize), y: Math.floor(canvas.height - y * tileSize), width: Math.ceil(tileSize) + 0.8, height: -(Math.ceil(tileSize) + 0.8) }, offscreenCtx); } } if (showGrid) { offscreenCtx.strokeStyle = "rgba(255, 255, 255, 0.2)"; offscreenCtx.lineWidth = 1; offscreenCtx.beginPath(); for (let x = 0; x <= grid.width; x++) { const xP = x * tileSize; offscreenCtx.moveTo(xP, 0); offscreenCtx.lineTo(xP, offscreenCanvas.height); } for (let y = 0; y
```

<= grid.height; y++) { const yP = offscreenCanvas.height - y * tileSize; offscreenCtx.moveTo(0, yP); offscreenCtx.lineTo(offscreenCanvas.width, yP); } offscreenCtx.stroke(); } } function drawWorld() { const cX = Math.floor(camera.x), cY = Math.floor(camera.y); if (cX !== lastCameraX || cY !== lastCameraY || grid.width !== lastGridWidth) { worldDirty = true; lastCameraX = cX; lastCameraY = cY; lastGridWidth = grid.width; } if (worldDirty) { renderWorldToBuffer(); worldDirty = false; } ctx.drawImage(offscreenCanvas, 0, 0); } function getBlockCache(x, y) { if (worldCache[x]) return worldCache[x][y]; } function getBlockObject(states) { return (blockData[states.type] || renderers.default)(states); } function drawMobs() { if (typeof mbwom === 'undefined' || !mbwom.mobs) return; for (let key in mbwom.mobs) { try { const mob = mbwom.mobs[key]; if (!mob || !mob.type) { delete mbwom.mobs[key]; continue; } const sX = (Number(mob.x) - camera.x) * tileSize, sY = canvas.height - (-Number(mob.y) - camera.y) * tileSize; if (sX < -200 || sX > canvas.width + 200 || sY < -200 || sY > canvas.height + 200) continue; let mW = tileSize, mH = tileSize * 2; if (['chicken', 'slime', 'magmacube'].includes(mob.type)) { mW = tileSize; mH = tileSize; } else if (mob.type === 'enderdragon') { mW = tileSize * 24; mH = tileSize * 8; } else if (['nethereye', 'rabbit', 'bat'].includes(mob.type)) { mW = tileSize * 0.75; mH = tileSize * 0.75; } else if (['pig', 'wolf', 'spider', 'sheep'].includes(mob.type)) { mW = tileSize * 2; mH = tileSize * 1.2; } else if (mob.type === 'enderman') { mW = tileSize * 1.2; mH = tileSize * 3; } else if (mob.type === 'squid') { mW = tileSize; mH = tileSize * 2; } else if (['cow', 'cowctus cow', 'mushroom cow'].includes(mob.type)) { mW = tileSize * 2.6; mH = tileSize * 2; } let mobImg = window.images[mob.type]; if (mobImg?.complete && mobImg.naturalWidth > 0) { ctx.save(); ctx.translate(sX, sY); if (mob.direction === 1) ctx.scale(-1, 1); ctx.imageSmoothingEnabled = false; ctx.drawImage(mobImg, 0, 0, mobImg.naturalWidth, mobImg.naturalHeight, -(mW / 2), -mH, mW, mH); ctx.restore(); } else { let color = "rgba(255, 0, 0, 0.4)"; if (mob.type === 'zombie') color = "rgba(46, 125, 50, 0.5)"; else if (mob.type === 'skeleton') color = "rgba(224, 224, 224, 0.5)"; else if (mob.type === 'enderman') color = "rgba(49, 27, 146, 0.5)"; ctx.fillStyle = color; ctx.fillRect(sX - (mW / 2), sY - mH, mW, mH); } if (typeof selectedMob !== 'undefined' && mob === selectedMob && typeof currentTool !== 'undefined' && currentTool === 'move') { ctx.fillStyle = "rgba(0, 120, 215, 0.3)"; ctx.fillRect(sX - (mW / 2), sY - mH, mW, mH); ctx.strokeStyle = "#0078D7"; ctx.lineWidth = 2; ctx.strokeRect(sX - (mW / 2), sY - mH, mW, mH); ctx.lineWidth = 1; } ctx.fillStyle = "#FFFFFF"; ctx.font = "bold 12px Arial"; ctx.textAlign = "center"; ctx.shadowColor = "black"; ctx.shadowBlur = 4; ctx.fillText(mob.name ? String(mob.name) : String(mob.type).toUpperCase(), sX, sY - mH - 8); ctx.shadowBlur = 0; if (mob.health !== undefined) { let maxHp = mob.maxHealth || (typeof MOBS_HP_DB !== 'undefined' ? MOBS_HP_DB[mob.type] : 20) || 20; let hpPercent = Math.max(0, Math.min(1, Math.ceil(Number(mob.health)) / maxHp)); ctx.fillStyle = "rgba(0, 0, 0, 0.5)"; ctx.fillRect(sX - (mW/2), sY - mH - 4, mW, 4); ctx.fillStyle = hpPercent > 0.3 ? "#00FF00" : "#FF0000"; ctx.fillRect(sX - (mW/2), sY - mH - 4, mW * hpPercent, 4); } let myName = localStorage.getItem('mbw_username') || "Player"; if (mob.lockedBy && mob.lockedBy !== myName && mob.lastMoveTime && (Date.now() - mob.lastMoveTime < 500)) { ctx.fillStyle = "rgba(255, 0, 0, 0.3)"; ctx.fillRect(sX - (mW / 2), sY - mH, mW, mH); ctx.fillStyle = "#FF5555"; ctx.font = "bold 10px Arial"; ctx.textAlign = "center"; ctx.shadowColor = "black"; ctx.shadowBlur = 3; ctx.fillText("🔒 " + mob.lockedBy, sX, sY - mH - 22); ctx.shadowBlur = 0; } } catch (e) { console.error("Mob error"); } } } function drawPlayer() { if (typeof mbwom === 'undefined' || !mbwom.world || mbwom.world.worldX === undefined || mbwom.world.worldY === undefined) return; const pS = mbwom.world.scene || 1, cS = typeof mbwom.currentScene !== 'undefined' ? mbwom.currentScene : 1; if (pS !== cS) return; const sX = (Number(mbwom.world.worldX) - camera.x) * tileSize, sY = canvas.height - (-Number(mbwom.world.worldY) - camera.y) * tileSize; if (sX < -200 || sX > canvas.width + 200 || sY < -200 || sY > canvas.height + 200) return; const pW = tileSize * 1.4, pH = tileSize * 2.0; let pI = window.images['Player']; if (pI?.complete && pI.naturalWidth > 0) { ctx.save(); ctx.translate(sX, sY); ctx.imageSmoothingEnabled = false; ctx.drawImage(pI, 0, 0, pI.naturalWidth, pI.naturalHeight, -(pW / 2), -pH, pW, pH); ctx.restore(); } else { ctx.fillStyle = "rgba(0, 100, 255, 0.6)"; ctx.fillRect(sX - (pW / 2), sY - pH, pW, pH); ctx.fillStyle = "rgba(255, 200, 150, 0.8)"; ctx.fillRect(sX - (pW / 2), sY - pH, pW, pH * 0.3); ctx.strokeStyle = "#FFFFFF"; ctx.strokeRect(sX - (pW / 2), sY - pH, pW, pH); } ctx.fillStyle = "#00FFFF"; ctx.font = "bold 14px Arial"; ctx.textAlign = "center"; ctx.shadowColor = "black"; ctx.shadowBlur = 4; ctx.fillText("PLAYER", sX, sY - pH - 8); ctx.shadowBlur = 0; }

```
window.fillSelectionWithBucket = blockState => { if (!window.selection || !window.selection.type) return false; let upd = 0; if (window.selection.type === 'magic' && window.selection.points) { window.selection.points.forEach(p => { mbwom.setBlockState(p.x, p.y, blockState); if (typeof renderBlock === 'function') renderBlock(p.x, p.y); upd++; }); } else if (window.selection.type === 'rect') { const rcts = [...window.selection.subRects]; if (window.selection.p1 && window.selection.p2) rcts.push({p1: window.selection.p1, p2: window.selection.p2}); rcts.forEach(r => { const minX = Math.min(r.p1.x, r.p2.x), maxX = Math.max(r.p1.x, r.p2.x), minY = Math.min(r.p1.y, r.p2.y), maxY = Math.max(r.p1.y, r.p2.y); for (let x = minX; x <= maxX; x++) for (let y = minY; y <= maxY; y++) { mbwom.setBlockState(x, y, blockState); if (typeof renderBlock === 'function') renderBlock(x, y); upd++; } }); } if (upd > 0) { if (typeof worldDirty !== 'undefined') worldDirty = true; return true; } return false; }; function drawUI() { const cDiv = document.getElementById('coords-overlay'), sDiv = document.getElementById('selection-overlay'); if (cDiv) cDiv.innerText = `X: ${Math.floor(mouse.worldX)} Y: ${Math.floor(mouse.worldY)}`; if (sDiv) { if (window.selection?.type === 'rect' && window.selection.p1 && window.selection.p2) { sDiv.innerText = `W: ${Math.abs(window.selection.p2.x - window.selection.p1.x) + 1} H: ${Math.abs(window.selection.p2.y - window.selection.p1.y) + 1}`; sDiv.style.display = 'block'; } else sDiv.style.display = 'none'; } if (!['paste', 'select', 'lasso'].includes(currentTool)) { const size = typeof toolSize !== 'undefined' ? toolSize : 1; const oS = Math.floor(size / 2), oE = Math.floor((size - 1) / 2); ctx.strokeStyle = "#4DA6FF"; ctx.lineWidth = 2; ctx.fillStyle = "rgba(77, 166, 255, 0.2)"; const cX = -oS + (size - 1) / 2, cY = -oS + (size - 1) / 2, rSq = (size / 2) ** 2; const isInside = (dx, dy) => { if (dx < -oS || dx > oE || dy < -oS || dy > oE) return false; if (typeof toolRounded !== 'undefined' && toolRounded && size > 1) return ((dx - cX)**2 + (dy - cY)**2 <= rSq); return true; }; ctx.beginPath(); for (let dx = -oS; dx <= oE; dx++) for (let dy = -oS; dy <= oE; dy++) if (isInside(dx, dy)) ctx.fillRect((mouse.worldX + dx - camera.x) * tileSize, canvas.height - (mouse.worldY + dy - camera.y) * tileSize, tileSize, -tileSize); ctx.beginPath(); for (let dx = -oS; dx <= oE; dx++) { for (let dy = -oS; dy <= oE; dy++) { if (isInside(dx, dy)) { const dX = (mouse.worldX + dx - camera.x) * tileSize, dY = canvas.height - (mouse.worldY + dy - camera.y) * tileSize; if (!isInside(dx + 1, dy)) { ctx.moveTo(dX + tileSize, dY); ctx.lineTo(dX + tileSize, dY - tileSize); } if (!isInside(dx - 1, dy)) { ctx.moveTo(dX, dY); ctx.lineTo(dX, dY - tileSize); } if (!isInside(dx, dy + 1)) { ctx.moveTo(dX, dY - tileSize); ctx.lineTo(dX + tileSize, dY - tileSize); } if (!isInside(dx, dy - 1)) { ctx.moveTo(dX, dY); ctx.lineTo(dX + tileSize, dY); } } } } ctx.stroke(); } if (currentTool === 'paste' && window.clipboard) { ctx.save(); ctx.globalAlpha = 0.5; window.clipboard.data.forEach(bD => { if (!bD.state) return; drawBlock(getBlockObject(bD.state), { x: (mouse.worldX + bD.dx - camera.x) * tileSize, y: canvas.height - (mouse.worldY + bD.dy - camera.y) * tileSize, width: tileSize, height: -tileSize }); }); ctx.restore(); ctx.strokeStyle = "#00FFFF"; ctx.lineWidth = 2; ctx.strokeRect((mouse.worldX - camera.x) * tileSize, canvas.height - (mouse.worldY - camera.y) * tileSize, window.clipboard.width * tileSize, -(window.clipboard.height * tileSize)); } if (window.selection.type === 'magic' && window.selection.points?.length > 0) { ctx.fillStyle = "rgba(0, 255, 255, 0.2)"; ctx.strokeStyle = "#87CEFA"; ctx.lineWidth = 2; ctx.setLineDash([6, 6]); ctx.lineDashOffset = -(Date.now() / 50); ctx.beginPath(); window.selection.points.forEach(p => { const sX = (p.x - camera.x) * tileSize, sY = canvas.height - (p.y - camera.y) * tileSize; ctx.fillRect(sX, sY, tileSize, -tileSize); if (!window.selection.pointSet.has((p.x + 1) + "," + p.y)) { ctx.moveTo(sX + tileSize, sY); ctx.lineTo(sX + tileSize, sY - tileSize); } if (!window.selection.pointSet.has((p.x - 1) + "," + p.y)) { ctx.moveTo(sX, sY); ctx.lineTo(sX, sY - tileSize); } if (!window.selection.pointSet.has(p.x + "," + (p.y + 1))) { ctx.moveTo(sX, sY - tileSize); ctx.lineTo(sX + tileSize, sY - tileSize); } if (!window.selection.pointSet.has(p.x + "," + (p.y - 1))) { ctx.moveTo(sX, sY); ctx.lineTo(sX + tileSize, sY); } }); ctx.stroke(); ctx.setLineDash([]); } if (window.selection.type === 'rect') { const rToDraw = [...window.selection.subRects]; if (window.selection.p1 && window.selection.p2) rToDraw.push({ p1: window.selection.p1, p2: window.selection.p2 }); rToDraw.forEach(r => { const sX = (Math.min(r.p1.x, r.p2.x) - camera.x) * tileSize, sY = canvas.height - (Math.min(r.p1.y, r.p2.y) - camera.y) * tileSize; const sW = (Math.max(r.p1.x, r.p2.x) - Math.min(r.p1.x, r.p2.x) + 1) * tileSize, sH = -(Math.max(r.p1.y, r.p2.y) - Math.min(r.p1.y, r.p2.y) + 1) * tileSize; ctx.fillStyle = "rgba(0, 255, 255, 0.2)"; ctx.fillRect(sX, sY, sW, sH); ctx.strokeStyle = "#87CEFA"; ctx.lineWidth = 2; ctx.setLineDash([6, 6]); ctx.lineDashOffset = -(Date.now() / 50); ctx.strokeRect(sX, sY, sW, sH); ctx.setLineDash([]); }); }
```

```
if (window.selection.type === 'poly' && window.selection.path.length > 0) { ctx.strokeStyle = "#FFD700"; ctx.lineWidth = 2; ctx.beginPath(); window.selection.path.forEach((p, i) => { const sX = (p.x - camera.x) * tileSize, sY = canvas.height - (p.y - camera.y) * tileSize; if (i === 0) ctx.moveTo(sX, sY); else ctx.lineTo(sX, sY); }); if (!window.selection.dragging && window.selection.path.length > 2) { ctx.closePath(); ctx.fillStyle = "rgba(255, 215, 0, 0.1)"; ctx.fill(); } ctx.stroke(); } if (typeof currentTool !== 'undefined' && currentTool === 'spawn_mob' && typeof currentMobToSpawn !== 'undefined' && currentMobToSpawn) { let mI = window.images[currentMobToSpawn]; if (mI?.complete && mI.naturalWidth > 0) { ctx.save(); ctx.globalAlpha = 0.6; ctx.imageSmoothingEnabled = false; let tS = typeof tileSize !== 'undefined' ? tileSize : 16, dW = tS * 1.2; if (currentMobToSpawn === 'enderdragon') dW = tS * 6; if (['ghast', 'slime', 'magmacube'].includes(currentMobToSpawn)) dW = tS * 2.5; if (currentMobToSpawn === 'spider') dW = tS * 1.5; let dH = dW * (mI.naturalHeight / mI.naturalWidth); ctx.drawImage(mI, 0, 0, mI.naturalWidth, mI.naturalHeight, mouse.canvasX - (dW / 2), canvas.height - mouse.canvasY - dH, dW, dH); ctx.restore(); } } if (typeof window.mySpectators !== 'undefined' && window.mySpectators.size > 0) { ctx.save(); const txt = "👁 " + Array.from(window.mySpectators).join(", "); ctx.font = "bold 16px Arial"; const bW = ctx.measureText(txt).width + 30, bH = 34, sX = canvas.width - bW - 20, sY = 20; ctx.fillStyle = "rgba(0, 0, 0, 0.65)"; ctx.fillRect(sX, sY, bW, bH); ctx.strokeStyle = Math.floor(Date.now() / 500) % 2 === 0 ? "#ff7675" : "#e74c3c"; ctx.lineWidth = 2; ctx.strokeRect(sX, sY, bW, bH); ctx.fillStyle = "#ff7675"; ctx.textAlign = "center"; ctx.textBaseline = "middle"; ctx.fillText(txt, sX + (bW / 2), sY + (bH / 2)); ctx.restore(); } if (typeof currentTool !== 'undefined' && currentTool === 'pixelart' && window.pendingPixelArt?.imgCanvas) { ctx.save(); ctx.globalAlpha = 0.5; ctx.imageSmoothingEnabled = false; const sX = (Math.floor(mouse.worldX) - camera.x) * tileSize, dY = canvas.height - (Math.floor(mouse.worldY) - camera.y) * tileSize - tileSize; const dW = window.pendingPixelArt.width * tileSize, dH = window.pendingPixelArt.height * tileSize; ctx.drawImage(window.pendingPixelArt.imgCanvas, sX, dY, dW, dH); ctx.strokeStyle = "#FFD700"; ctx.lineWidth = 2; ctx.setLineDash([6, 6]); ctx.lineDashOffset = -(Date.now() / 50); ctx.strokeRect(sX, dY, dW, dH); ctx.restore(); } } function mainLoop() { cameraMovement(); mouse.calculateCoordinates(); if (!window.spectatingTargetId && typeof mineAndPlace === 'function') mineAndPlace(); drawWorld(); if (typeof drawMobs === 'function') drawMobs(); drawPlayer(); const ltToggle = document.getElementById('live-time-toggle'), tInp = document.getElementById('gr-time'); if (ltToggle?.checked && tInp) { if (typeof window.lastTimeUpdate === 'undefined') window.lastTimeUpdate = Date.now(); let ahora = Date.now(); if (ahora - window.lastTimeUpdate >= 12000) { tInp.value = Number(tInp.value) + 1 > 99 ? 0 : Number(tInp.value) + 1; window.lastTimeUpdate = ahora; } let prog = Number(tInp.value) / 99, osc = 0, oscM = 0.75; if (prog > 0.45 && prog <= 0.5) osc = ((prog - 0.45) / 0.05) * oscM; else if (prog > 0.5 && prog <= 0.9) osc = oscM; else if (prog > 0.9) osc = oscM - (((prog - 0.9) / 0.1) * oscM); if (osc > 0) { ctx.save(); ctx.fillStyle = `rgba(5, 5, 20, ${osc})`; ctx.fillRect(0, 0, canvas.width, canvas.height); ctx.restore(); } } if (typeof drawUI === 'function') drawUI(); if (typeof isMultiplayer !== 'undefined' && isMultiplayer && window.networkCursors) { let ahora = Date.now(); for (let autor in window.networkCursors) { let cur = window.networkCursors[autor]; if (ahora - cur.lastUpdate > 60000) continue; let sX = (cur.x - camera.x) * tileSize, sY = canvas.height - ((cur.y - camera.y) * tileSize) - tileSize; ctx.fillStyle = "rgba(77, 166, 255, 0.4)"; ctx.fillRect(sX, sY, tileSize, tileSize); ctx.strokeStyle = "#4DA6FF"; ctx.strokeRect(sX, sY, tileSize, tileSize); ctx.fillStyle = "white"; ctx.font = "16px 'Pixeltype', sans-serif"; ctx.textAlign = "center"; ctx.shadowColor = "black"; ctx.shadowBlur = 4; ctx.fillText(autor, sX + (tileSize/2), sY - 5); ctx.shadowBlur = 0; ctx.save(); ctx.translate(sX + (tileSize / 2), sY + (tileSize / 2)); ctx.beginPath(); ctx.moveTo(0, 0); ctx.lineTo(0, 17); ctx.lineTo(4, 13); ctx.lineTo(8, 20); ctx.lineTo(11, 18); ctx.lineTo(7, 12); ctx.lineTo(13, 12); ctx.closePath(); ctx.fillStyle = "rgba(255, 255, 255, 0.85)"; ctx.fill(); ctx.lineWidth = 1; ctx.strokeStyle = "rgba(0, 0, 0, 0.7)"; ctx.stroke(); ctx.restore(); } } requestAnimationFrame(mainLoop); } function changeDimension(sceneIndex) { if (typeof mbwom !== 'undefined' && mbwom.world && mbwom.world["scene" + sceneIndex]) { mbwom.loadScene(sceneIndex); mbwom.currentScene = sceneIndex; initializeWorldCache(); closeModal('dimensions-modal'); const ico = document.getElementById('current-dim-icon'); if (ico) ico.src = sceneIndex === 1 ? "assets/Overworld icon.png" : sceneIndex === 2 ? "assets/Nether icon.png" : "assets/End icon.png"; } else alert("This dimension is not generated in the current world."); } let
```

```
multiplayerPlayerLimit = 2; function setMpView(viewName) { ['mp-view-home', 'mp-view-create-1', 'mp-view-create-2', 'mp-view-join', 'multiplayer-status', 'mp-shared-profile'].forEach(id => document.getElementById(id).style.display = 'none'); const t = document.getElementById('mp-modal-title'); if (viewName === 'home') { t.innerText = "🌐 Multiplayer"; document.getElementById('mp-view-home').style.display = 'flex'; } else if (viewName === 'create-1') { t.innerText = "Create Server"; document.getElementById('mp-shared-profile').style.display = 'flex'; document.getElementById('mp-view-create-1').style.display = 'block'; loadMpProfile(); } else if (viewName === 'create-2') { t.innerText = "Server Active"; document.getElementById('mp-view-create-2').style.display = 'block'; document.getElementById('multiplayer-status').style.display = 'block'; } else if (viewName === 'join') { t.innerText = "Join Server"; document.getElementById('mp-shared-profile').style.display = 'flex'; document.getElementById('mp-view-join').style.display = 'block'; document.getElementById('multiplayer-status').style.display = 'block'; loadMpProfile(); if (typeof loadPublicServers === 'function') loadPublicServers(); } } function setMpLimit(limit) { multiplayerPlayerLimit = limit; document.querySelectorAll('.mp-limit-btn').forEach(btn => btn.classList.remove('selected-limit')); document.querySelectorAll('.mp-limit-btn')[limit - 1].classList.add('selected-limit'); } function copyRoomId() { const id = document.getElementById('my-peer-id'); if (id.value.trim() !== "") navigator.clipboard.writeText(id.value).then(() => { const h = document.getElementById('copy-hint'); h.style.visibility = 'visible'; setTimeout(() => h.style.visibility = 'hidden', 2000); }); } window.openUnifiedProfile = () => { openModal('unified-profile-modal'); document.getElementById('modal-global-pfp').src = localStorage.getItem('mbw_profile_pic') || "assets/default pfp.png"; document.getElementById('global-profile-username').value = localStorage.getItem('mbw_username') || "Player"; }; window.handleGlobalProfileUpload = e => { const f = e.target.files[0]; if (f) { const r = new FileReader(); r.onload = ev => { const b64 = ev.target.result; localStorage.setItem('mbw_profile_pic', b64); document.getElementById('modal-global-pfp').src = b64; const tImg = document.getElementById('top-profile-img'); if (tImg) tImg.src = b64; const mpPrev = document.getElementById('mp-pfp-preview'); if (mpPrev) mpPrev.style.backgroundImage = `url(${b64})`; if (typeof myPresenceRef !== 'undefined' && myPresenceRef) myPresenceRef.update({ pfp: b64 }); }; r.readAsDataURL(f); } }; window.handleGlobalUsernameChange = nName => { let clName = nName.trim() || 'Player'; localStorage.setItem('mbw_username', clName); const mpInp = document.getElementById('mp-username-input'); if (mpInp) mpInp.value = clName; if (typeof myPresenceRef !== 'undefined' && myPresenceRef) myPresenceRef.update({ username: clName }); }; window.addEventListener('DOMContentLoaded', () => { const tImg = document.getElementById('top-profile-img'); if (tImg) tImg.src = localStorage.getItem('mbw_profile_pic') || "assets/default pfp.png"; document.body.addEventListener('click', e => { if (e.target && (['mp-pfp-preview', 'mp-username-input'].includes(e.target.id) || e.target.closest('#mp-pfp-preview'))) { e.preventDefault(); openUnifiedProfile(); } }); }); window.toggleHelpMenu = () => document.getElementById("help-dropdown").classList.toggle("show"); window.closeHelpMenu = () => { let d = document.getElementById("help-dropdown"); if (d?.classList.contains('show')) d.classList.remove("show"); }; document.addEventListener('click', e => { if (!e.target.matches('.icon-btn') && !e.target.closest('.dropdown-container')) window.closeHelpMenu(); }); const canvasElement = document.getElementById('canvas'); if (canvasElement) { canvasElement.addEventListener('wheel', e => { e.preventDefault(); let mX = typeof mouse !== 'undefined' && mouse.canvasX !== null ? mouse.canvasX : canvasElement.width / 2; let mY = typeof mouse !== 'undefined' && mouse.canvasY !== null ? mouse.canvasY : canvasElement.height / 2; let eWX = camera.x + (mX / tileSize), eWY = camera.y + (mY / tileSize); let step = currentZoom < 50 ? 5 : (currentZoom < 200 ? 10 : 25); let nZoom = Math.max(2, Math.min(600, currentZoom + (e.deltaY < 0 ? step : -step))); if (nZoom !== currentZoom) { let nTS = BASE_TILE_SIZE * (nZoom / 100); camera.x = eWX - (mX / nTS); camera.y = eWY - (mY / nTS); applyZoomExact(nZoom); if (typeof mouse !== 'undefined' && mouse.canvasX !== null) { mouse.gridX = Math.floor(mouse.canvasX / tileSize); mouse.gridY = Math.floor(mouse.canvasY / tileSize); mouse.calculateCoordinates(); } if (typeof worldDirty !== 'undefined') worldDirty = true; } }, { passive: false }); } window.toggleAddonsMenu = () => { const m = document.getElementById('addons-dropdown'), b = document.getElementById('addons-
```

dropdown-container').querySelector('.tab-btn'); m.classList.toggle('show'); if (m.classList.contains('show')) b.classList.add('active'); else b.classList.remove('active'); }; const skinDB = { dbName: "MBW_SkinsDB", storeName: "skins", init: function() { return new Promise((res, rej) => { let req = indexedDB.open(this.dbName, 1); req.onupgradeneeded = e => { let db = e.target.result; if (!db.objectStoreNames.contains(this.storeName)) db.createObjectStore(this.storeName); }; req.onsuccess = e => res(e.target.result); req.onerror = e => rej(e.target.error); }); }, saveSkin: async function(id, b64) { let db = await this.init(); return new Promise((res, rej) => { let tx = db.transaction(this.storeName, "readwrite"); tx.objectStore(this.storeName).put({ data: b64, time: Date.now() }, String(id)); tx.oncomplete = () => res(); tx.onerror = () => rej(); }); }, getSkin: async function(id) { let db = await this.init(); return new Promise(res => { let req = db.transaction(this.storeName, "readonly").objectStore(this.storeName).get(String(id)); req.onsuccess = () => res(req.result ? (req.result.data || req.result) : null); req.onerror = () => res(null); }); }, getAllSorted: async function() { let db = await this.init(); return new Promise(res => { let tx = db.transaction(this.storeName, "readonly"), store = tx.objectStore(this.storeName), reqV = store.getAll(), reqK = store.getAllKeys(); tx.oncomplete = () => { let vals = reqV.result || [], keys = reqK.result || []; res(keys.map((k, i) => ({ id: k, time: vals[i].time || 0, base64: vals[i].data || vals[i] })).sort((a, b) => b.time - a.time)); }; tx.onerror = () => res([]); }); } }; window.skinCache = window.skinCache || {}; window.skinFetchInProgress = window.skinFetchInProgress || {}; window.getSpawnskinImage = skinID => { if (window.skinCache[skinID]) return window.skinCache[skinID]; if (!window.skinFetchInProgress[skinID]) { window.skinFetchInProgress[skinID] = true; const IImg = new Image(); IImg.onload = () => { window.skinCache[skinID] = IImg; if (typeof worldDirty !== 'undefined') worldDirty = true; }; IImg.onerror = () => skinDB.getSkin(skinID).then(b64 => { if (b64) { const img = new Image(); img.onload = () => { if (typeof worldDirty !== 'undefined') worldDirty = true; }; img.src = b64; window.skinCache[skinID] = img; } else { window.skinCache[skinID] = { hasFailed: true }; if (typeof worldDirty !== 'undefined') worldDirty = true; } }); IImg.src = `assets/spawnskins/${skinID}.png`; } return null; }; window.openSpawnskinsModal = () => { if (typeof openModal === 'function') openModal('spawnskins-addon-modal'); refreshLoadedSkinsList(); }; window.closeSpawnskinsModal = () => { if (typeof closeModal === 'function') closeModal('spawnskins-addon-modal'); const lD = document.getElementById('loaded-skins-list'); if (lD) lD.innerHTML = ""; }; window.refreshLoadedSkinsList = async () => { const lD = document.getElementById('loaded-skins-list'); if (!lD) return; lD.innerHTML = "

Buscando...

"; try { const skins = await skinDB.getAllSorted(); lD.innerHTML = ""; if (skins.length === 0) return lD.innerHTML = "

Ninguna skin cargada aún

"; skins.forEach(s => { const itm = document.createElement('div'); itm.style.cssText = `background: #fff; border: 1px solid #555; padding: 4px 6px; font-size: 11px; font-family: monospace; color: #000; border-radius: 3px; font-weight: bold; display: flex; justify-content: space-between; align-items: center;`; itm.innerHTML = `📄 ${s.id}.png

`; lD.appendChild(itm); }); } catch (e) { lD.innerHTML = "

Error al cargar lista

"; } }; window.importLocalSkins = e => { const fL = e.target.files; if (!fL || fL.length === 0) return; let cnt = 0; for (let f of fL) { const id = f.name.replace(/\.[^/.]+$/, ""), r = new FileReader(); r.onload = async ev => { const b64 = ev.target.result; await skinDB.saveSkin(id, b64); const img = new Image(); img.onload = () => { if (typeof worldDirty !== 'undefined') worldDirty = true; }; img.src = b64; window.skinCache[id] = img; if (++cnt === fL.length) { document.getElementById('skin-upload-input').value = ""; refreshLoadedSkinsList(); } }; r.readAsDataURL(f); } }; window.processSpawnskinUpload = e => { const f = e.target.files[0], l = document.getElementById('loaded-skins-list'); if (!f) return; if (l) l.innerHTML = "*Procesando archivo...*"; const r = new FileReader(); r.onload = ev => { let enc = ev.target.result, wd = null; try { wd = JSON.parse(enc); } catch (err) { try { let dec = ""; for (let a = 0, b = enc.length; a < b;) { let

c = a++; let cC = enc.charCodeAt(c) - (c * 5 % 33 + 1); dec += String.fromCodePoint(Math.max(0, cC)); } wd = JSON.parse(dec); } catch (err2) { if (l) l.innerHTML = "**Error: Archivo corrupto.**"; e.target.value = ""; return; } } let found = [], proc = new Set(); if (wd) [wd.mobs1, wd.mobs2, wd.mobs3].forEach(g => { if (g) for (let k in g) { let m = g[k]; if (m.type === "spawnskin" && m.skin && !proc.has(m.skin)) { proc.add(m.skin); found.push({ name: m.name || "Sin nombre", skinId: m.skin }); } } }); if (found.length > 0) { window.editorImportedSkins = found; let htm = `

Skins importadas: ${found.length}

`; found.forEach(s => htm += `

${s.name}
ID: ${s.skinId}

`); if (l) l.innerHTML = htm; } else if (l) l.innerHTML = "*No se encontraron skins.*"; e.target.value = ""; }; r.readAsText(f); }; window.openPixelArtModal = () => { if (typeof openModal === 'function') openModal('pixelart-addon-modal'); }; const blockPalette = { "cloth_white": [225, 225, 225], "cloth_lightgray": [160, 160, 160], "cloth_gray": [85, 85, 85], "cloth_black": [25, 25, 25], "cloth_brown": [85, 55, 30], "cloth_purple": [120, 50, 155], "cloth_magenta": [185, 65, 175], "cloth_red": [165, 45, 45], "cloth_orange": [225, 115, 35], "cloth_pink": [235, 140, 165], "cloth_yellow": [225, 200, 45], "cloth_lightgreen": [115, 185, 25], "cloth_green": [65, 105, 35], "cloth_cyan": [45, 115, 135], "cloth_lightblue": [105, 155, 210], "cloth_blue": [45, 60, 150], "cloth_rainbow": [200, 200, 200], "br": [80, 80, 80], "r": [120, 120, 120], "cs": [105, 105, 105], "ms": [95, 115, 80], "ms1": [95, 115, 80], "ms2": [95, 115, 80], "dt": [134, 96, 67], "dt_1": [110, 140, 65], "farm": [102, 70, 46], "myc": [111, 99, 105], "gdt": [119, 85, 58], "cdt": [119, 85, 58], "sb": [218, 210, 158], "sd": [218, 210, 158], "ss": [215, 205, 150], "ssd": [215, 205, 150], "gv": [130, 125, 120], "cy1": [160, 166, 179], "snowblock": [240, 250, 250], "ice": [160, 200, 255], "fice": [160, 200, 255], "fice_1": [160, 200, 255], "fice_2": [160, 200, 255], "fice_3": [160, 200, 255], "fice_4": [160, 200, 255], "wp": [160, 130, 80], "wd1": [100, 80, 50], "wd_1": [100, 80, 50], "wd_2": [100, 80, 50], "fw1": [100, 80, 50], "fw2": [100, 80, 50], "j": [150, 110, 80], "hai_1": [200, 180, 40], "hay": [200, 170, 30], "hay_1": [200, 170, 30], "hay_2": [200, 170, 30], "lv": [65, 125, 45], "lv1": [70, 110, 50], "lv2": [50, 90, 60], "lv3": [80, 130, 40], "lv4": [60, 100, 40], "lgr": [130, 200, 40], "pk": [220, 140, 30], "jl": [250, 150, 30], "mel": [100, 140, 50], "lemonb": [245, 230, 70], "bricks": [145, 80, 70], "books": [115, 90, 55], "bbb": [220, 220, 210], "dsb": [55, 55, 60], "clore": [90, 90, 90], "in": [135, 130, 125], "gd": [140, 135, 120], "dmore": [110, 130, 135], "rs": [130, 90, 90], "os": [25, 20, 35], "lap": [90, 100, 130], "egem": [100, 130, 100], "to": [140, 125, 90], "ib": [230, 230, 230], "gb": [248, 224, 70], "db": [99, 219, 213], "lapb": [39, 67, 138], "clb": [20, 20, 20], "top": [255, 180, 50], "tob": [255, 165, 45], "ob": [25, 15, 35], "n": [110, 55, 55], "nb": [45, 20, 25], "rnb": [95, 30, 35], "magma": [210, 95, 30], "glow": [235, 195, 100], "light": [255, 240, 180], "coral": [230, 115, 145], "es": [220, 225, 165], "pf": [165, 115, 165], "portalstone": [55, 115, 120], "bdcloth_white": [112, 112, 112], "bdcloth_black": [12, 12, 12], "bddt": [67, 48, 33], "bdr": [60, 60, 60], "bdcs": [52, 52, 52], "bdbbb": [110, 110, 105], "bdbricks": [72, 40, 35], "bdbooks": [57, 45, 27], "bdsb": [109, 101, 80], "bdwp": [80, 65, 40], "bdnb": [22, 10, 12], "bb": [57, 45, 27], "bdob": [12, 7, 18] }; function getClosestBlock(r, g, b) { let cB = "cloth_white", mD = Infinity; for (let block in blockPalette) { let [br, bg, bb] = blockPalette[block], dist = Math.sqrt((r - br)**2 + (g - bg)**2 + (b - bb)**2); if (dist < mD) { mD = dist; cB = block; } } return cB; } window.processPixelArt = e => { const f = e.target.files[0]; if (!f) return; const maxW = parseInt(document.getElementById('pixelart-max-width').value) || 64, r = new FileReader(); r.onload = ev => { const img = new Image(); img.onload = () => { let sc = img.width > maxW ? maxW / img.width : 1; const w = Math.floor(img.width * sc), h = Math.floor(img.height * sc), c = document.createElement('canvas'); c.width = w; c.height = h; const ctxT = c.getContext('2d'); ctxT.drawImage(img, 0, 0, w, h); window.pendingPixelArt = { data: ctxT.getImageData(0, 0, w, h).data, width: w, height: h, imgCanvas: c }; if (typeof currentTool !== 'undefined') currentTool = 'pixelart'; alert("¡Imagen lista! Cierra esta ventana y HAZ CLIC en cualquier lugar del mapa para construirla."); if (typeof closeModal === 'function') closeModal('pixelart-addon-modal'); document.getElementById('pixelart-upload').value = ""; }; img.src = ev.target.result; }; r.readAsDataURL(f); }; window.buildPixelArtInWorld = (pD, w, h, sX, sY) => { if (typeof historyManager !== 'undefined') historyManager.startAction(); for (let y = 0; y < h; y++) for (let x = 0; x < w; x++) { const i = (y * w + x) * 4; if (pD[i + 3] < 50) continue; const bT =

```
getClosestBlock(pD[i], pD[i + 1], pD[i + 2]), bX = Math.floor(sX + x), bY = Math.floor(sY - y); if (typeof mbwom !== 'undefined' && typeof mbwom.setBlockState === 'function') { const cS = mbwom.getBlockState(bX, bY), nS = { type: bT }; if (typeof historyManager !== 'undefined') historyManager.recordChange(bX, bY, cS, nS); mbwom.setBlockState(bX, bY, nS); if (typeof renderBlock === 'function') renderBlock(bX, bY); } } if (typeof worldDirty !== 'undefined') worldDirty = true; if (typeof historyManager !== 'undefined') historyManager.commitAction(); }; function setupPixelArtClicker() { const mC = document.getElementById('canvas') || document.querySelector('canvas'); if (!mC) return; mC.addEventListener('mousedown', e => { if (window.pendingPixelArt && typeof currentTool !== 'undefined' && currentTool === 'pixelart') { buildPixelArtInWorld(window.pendingPixelArt.data, window.pendingPixelArt.width, window.pendingPixelArt.height, Math.floor(mouse.worldX), Math.floor(mouse.worldY)); window.pendingPixelArt = null; if (typeof selectTool === 'function') selectTool('move'); else currentTool = 'move'; alert("¡Pixel Art pegado con éxito!"); } }); } if (document.readyState === "loading") document.addEventListener("DOMContentLoaded", setupPixelArtClicker); else setupPixelArtClicker(); window.addEventListener('click', e => { const aC = document.getElementById('addons-dropdown-container'), aM = document.getElementById('addons-dropdown'); if (aC && !aC.contains(e.target) && aM?.classList.contains('show')) { aM.classList.remove('show'); aC.querySelector('.tab-btn')?.classList.remove('active'); } const cT = e.target.closest('.tab-btn'); if (cT && cT.id !== 'btn-world-info') { const sb = document.getElementById('world-info-sidebar'); if (sb?.style.display === 'flex') { sb.style.display = 'none'; document.getElementById('btn-world-info')?.classList.remove('active'); const zC = document.getElementById('zoom-floating'); if (zC) zC.style.right = '20px'; } } }); window.addEventListener('keydown', e => { if (['inventory-search', 'ci-preview-title', 'custom-chest-name'].includes(e.target.id)) e.stopPropagation(); }, true); const toolSizeSlider = document.getElementById('tool-size-slider'), toolSizeDisplay = document.getElementById('tool-size-display'); function handleWheelZoomSize(e) { e.preventDefault(); let cV = parseInt(toolSizeSlider.value); e.deltaY < 0 ? cV++ : cV--; if (typeof updateToolSize === 'function') updateToolSize(cV, 'blur'); } if (toolSizeSlider) toolSizeSlider.addEventListener('wheel', handleWheelZoomSize, { passive: false }); if (toolSizeDisplay) toolSizeDisplay.addEventListener('wheel', handleWheelZoomSize, { passive: false });
```